Charts & Dashboards.

Yesterday your data became a table. Today it becomes a picture. Line charts, bar charts, Matplotlib, and the dashboard pattern that ties it all together.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

7 of 24

AGENDA · DAY 07

How we'll spend our 90 minutes today.

Four chart types, one styling lesson, one cheatsheet, and a full dashboard exercise that combines everything you have learned in Phase 2.

DURATION
90 min

`00:00 – 00:10`

Day 6 Recap & Q&A

pandas, DataFrames, filtering, the student dashboard. Quick check.

`00:10 – 00:25`

st.line_chart + st.bar_chart

The two built-in Streamlit chart commands. Pandas direct, no setup.

`00:25 – 00:40`

Matplotlib via st.pyplot()

The industry standard. Full control over titles, labels, colors, sizing.

`00:40 – 00:55`

Chart Styling + Cheatsheet

How to make charts look intentional. Plus a printable cheatsheet for choosing the right chart.

`00:55 – 01:15`

Exercise: Full Student Dashboard

Sidebar filters + `st.dataframe` + `st.metric` + `st.bar_chart`. The Phase-2 capstone pattern.

`01:15 – 01:30`

Q&A and Day 8 Preview

Open floor, then a look at tomorrow's solo challenge.

RECAP · 10 MIN

Yesterday your data became a table. Today it becomes a chart.

Quick check that Day 6's student dashboard runs. If it doesn't, we fix it now — today's exercise builds directly on it.

WHAT WE COVERED YESTERDAY — DAY 06

- **The DataFrame** — rows have an index, columns have names
- **Loading data** — `pd.read_csv`, `read_excel`, `read_json`
- **Inspecting** — `head()`, `info()`, `describe()`
- **Filtering** — boolean masks, AND/OR/contains/isin
- **Built a filterable student dashboard** — sidebar + pandas + `st.dataframe`

CHECK YOUR LAPTOP NOW — VERIFY

```
# You should have Day 6's dashboard files:
$ ls day6/
students.csv
09_dashboard_solution.py

# Run the dashboard — should open in browser:
$ streamlit run 09_dashboard_solution.py
# → Browser opens at localhost:8501
# → Sidebar with branch/city filters
# → Filtered table + average marks
```

⚠ **IF THE DASHBOARD DOESN'T RUN** — raise your hand. Common issue: forgot to activate `.venv`, or `students.csv` is missing. Both fixable in 2 minutes.

A table shows data. A chart shows the story.

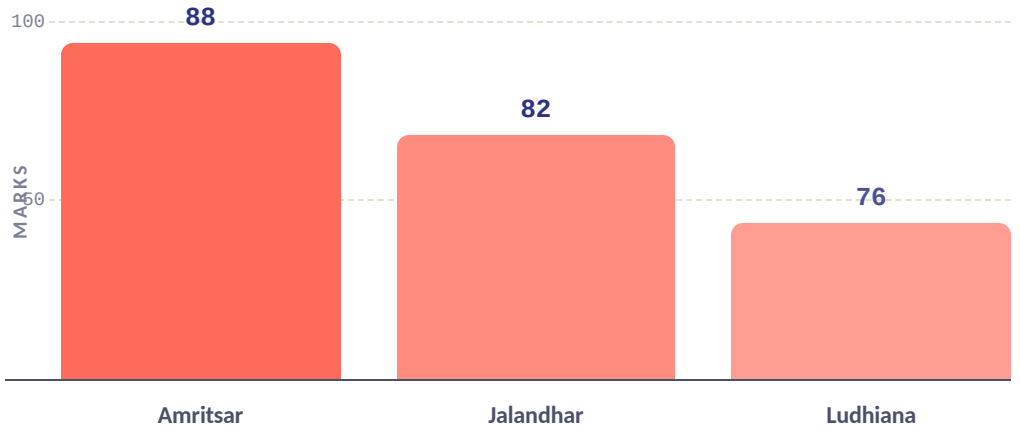
You can read 28 rows of marks. You cannot see the pattern. Charts make patterns visible in 5 seconds — the difference between data and insight.

THE TABLE — WHAT YOU BUILT YESTERDAY

STUDENT	MARKS
Aarav	85
Priya	92
Rahul	78
Meera	88
Sanjana	91
Vikram	75
Anjali	89
Karan	82

Can you tell which city has the highest average marks? **Give up? Exactly.**

THE CHART — WHAT YOU BUILD TODAY



Average marks by city. The pattern is obvious in 2 seconds. **That's the point.**

KEY IDEA Charts are not decoration. They are a **compression algorithm** — they take 28 rows and turn them into one shape your eye reads instantly. Use them whenever you want to show a pattern, not a value.

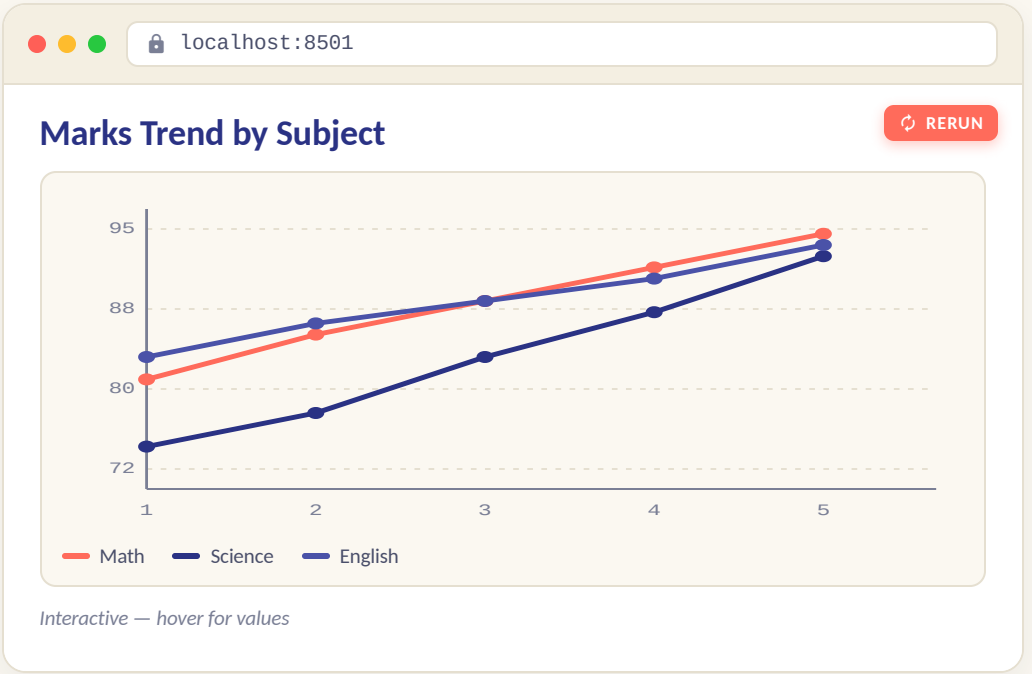
st.line_chart — the simplest chart in Streamlit.

Pass a pandas DataFrame with numeric columns. Streamlit draws a line chart. No setup, no configuration, no Matplotlib. **One line of code.**

app.py — 1 LINE OF CODE — PYTHON

```python
import streamlit as st
import pandas as pd

st.title("Marks Trend by Subject")

# Sample data — marks across 5 subjects
data = pd.DataFrame({
    "Math": [78, 82, 85, 88, 91],
    "Science": [72, 75, 80, 84, 89],
    "English": [80, 83, 85, 87, 90],
}, index=[1, 2, 3, 4, 5])  # test number

# ONE LINE — that's it
st.line_chart(data)


# That's the whole chart.
# Streamlit picks colors automatically,
# adds a legend, axes, and hover tooltips.
```

BROWSER PREVIEW · WHAT THE STUDENT SEES



Each column in the DataFrame became one line. The index became the x-axis.

1. **Pass a DataFrame** — Each numeric column becomes a line. The index becomes the x-axis.
2. **Colors are automatic** — Streamlit picks a palette. You can override with `color` arg (Streamlit 1.31+).
3. **Interactive by default** — Hover for values, drag to zoom, double-click to reset. No extra code.

USE WHEN You want to show how something changes over time (test scores, sales, temperature). The x-axis is usually time or sequence.

st.bar_chart — compare values across categories.

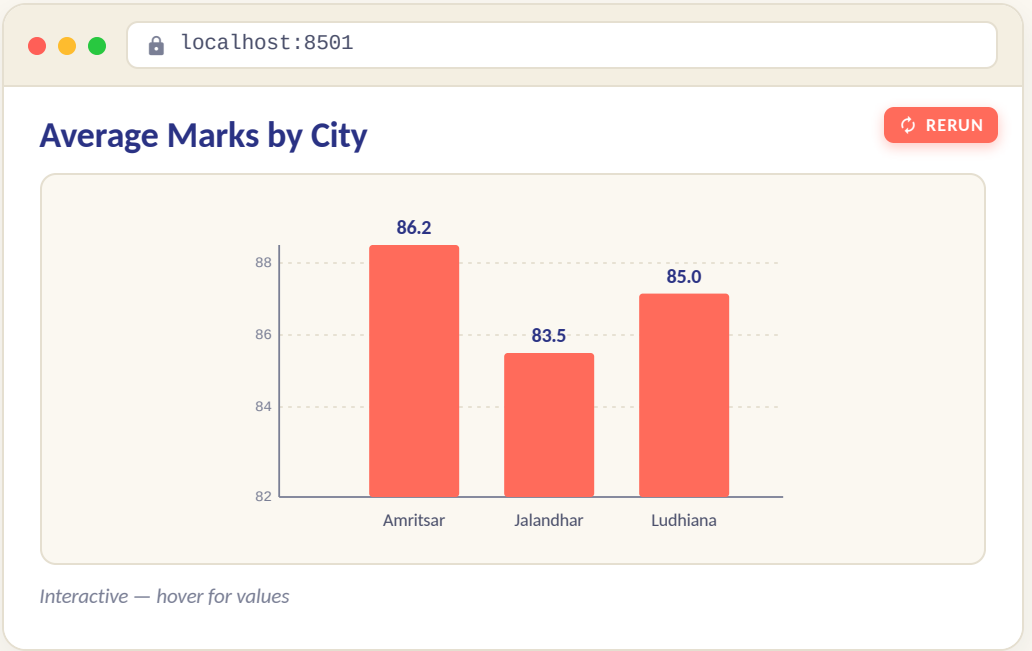
Same simplicity as `line_chart`, but for bars. Use when you want to compare quantities across distinct categories — cities, branches, products.

app.py GROUPBY + CHART PYTHON

```python
import streamlit as st
import pandas as pd

st.title("Average Marks by City")

# Aggregate: average marks per city
df = pd.read_csv("students.csv")
avg_by_city = df.groupby("city")["marks"].mean()

# ONE LINE — bar chart
st.bar_chart(avg_by_city)


# groupby + bar_chart = the dashboard pattern.
# Day 6's pandas + Day 7's charts = real analytics.
```

1. **groupby first** — Use pandas to aggregate, then chart the result. The chart shows the summary, not the raw data.
2. **Series or DataFrame** — Pass a Series (one bar per index) or a DataFrame (grouped bars per row).
3. **Same interactivity** — Hover, zoom, reset — all built-in, same as `line_chart`.

BROWSER PREVIEW · WHAT THE STUDENT SEES



Each row of the aggregated Series became one bar. The index became the x-axis.

USE WHEN You want to compare quantities across distinct categories. Cities, branches, products, months. Bars make the ranking obvious.

st.area_chart — line charts with filled volume.

Like a line chart, but the area under each line is filled. Good for showing **cumulative totals** or the **composition of a whole** over time.

WHEN TO USE AREA

Cumulative totals

Revenue over the year, expenses month by month. The filled area emphasizes the running total.

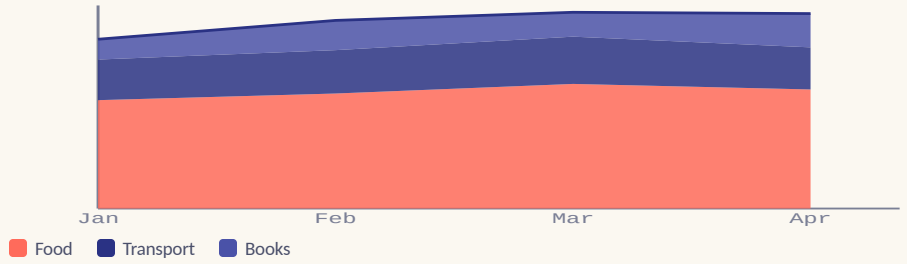
Stacked composition

Multiple series stacked, showing how each contributes to the whole. Like a pie chart that changes over time.

Volume emphasis

When the SIZE of the thing matters as much as the trend. Population, traffic, storage used.

STACKED AREA · FOOD + TRANSPORT + BOOKS



app.py — SAME API AS LINE — PYTHON

```python
import streamlit as st
import pandas as pd

# Monthly expenses by category
expenses = pd.DataFrame({
    "Food": [8000, 8500, 9200, 8800],
    "Transport": [3000, 3200, 3500, 3100],
    "Books": [1500, 2200, 1800, 2500],
}, index=["Jan", "Feb", "Mar", "Apr"])

st.area_chart(expenses)


# Same API as line_chart.
# Just a different visual emphasis.
```

The DataFrame is identical to what you'd pass to `line_chart` — the only change is the function name.

QUICK NOTE

Streamlit stacks the areas by default. Reading exact values is harder than on a line chart — that's the trade-off.

USE WHEN The volume matters as much as the trend. Skip if you just want a line chart — area charts can be harder to read exact values from.

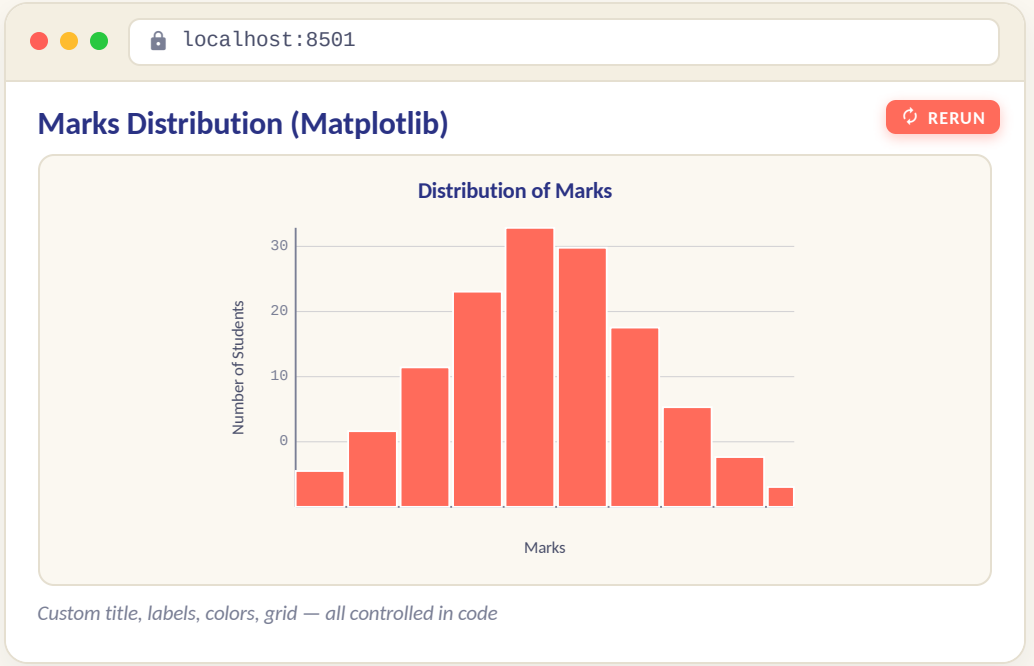
Matplotlib + st.pyplot() — when you need full control.

The built-in charts are easy but limited. Matplotlib gives you titles, labels, colors, sizing, multiple subplots, annotations — everything. The price: **10 lines instead of 1.**

app.py — 10 LINES, FULL CONTROL — PYTHON

```python
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

st.title("Marks Distribution (Matplotlib)")

df = pd.read_csv("students.csv")

# 1. Create a figure and axis
fig, ax = plt.subplots(figsize=(8, 4))

# 2. Plot on the axis
ax.hist(df["marks"], bins=10, color="#FF6B5B", edgecolor="white")

# 3. Customize
ax.set_title("Distribution of Marks", fontsize=14, color="#2A3284")
ax.set_xlabel("Marks")
ax.set_ylabel("Number of Students")
ax.grid(axis="y", alpha=0.3)

# 4. Pass the FIGURE to Streamlit
st.pyplot(fig)


# 10 lines instead of 1 — but full control.
```

BROWSER PREVIEW · WHAT THE STUDENT SEES



Custom title, labels, colors, grid — all controlled in code

Same data, 10 lines of code, presentation-grade chart. Note the coral fill — that's our brand color.

1. **`fig, ax = plt.subplots()`** — The Matplotlib pattern. `fig` is the whole figure, `ax` is one plot inside it. Plot on `ax`, pass `fig` to Streamlit.

USE WHEN You need titles, custom colors, multiple subplots, annotations, or a specific style. The built-in charts are for quick looks; Matplotlib is for presentations.

3. **`st.pyplot(fig)`** — Pass the figure object, not the pyplot module. Streamlit renders it as an image in the app.

Three ways to show data. Pick the right one.

Not everything is a chart. Sometimes you want a table, sometimes a single number, sometimes a full interactive grid. **Know the difference.**

`st.dataframe(df)`

Interactive table. Sortable, scrollable, resizable columns.

```
st.dataframe(df)
```

USE WHEN

You have a full DataFrame and want users to explore it. **Default for tables.**

DON'T USE WHEN

You have 1-2 numbers to show — use `st.metric` instead.

`st.table(df)`

Static table. Renders as-is, no interactivity.

```
st.table(df.head(10))
```

USE WHEN

You want a fixed view that doesn't change — for a report, a printout, or a small table where sorting would distract.

DON'T USE WHEN

The table has more than 20 rows — users can't scroll. Use `st.dataframe`.

`st.metric(label, value)`

A single big number with a label. The dashboard KPI card.

```
st.metric(label="Avg Marks",
          value="84.7",
          delta="+2.3")
```

USE WHEN

You have one important number to highlight — average, total, count, percentage. **The headline of your dashboard.**

DON'T USE WHEN

You have multiple rows — use a table or chart.

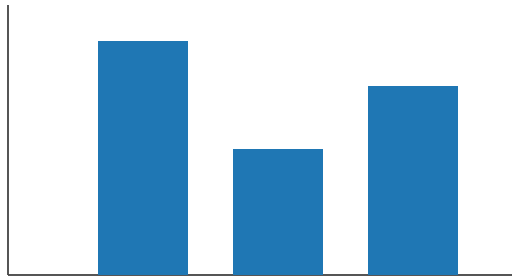
PATTERN Lead with `st.metric` for the headline numbers, follow with `st.dataframe` for the detail, use charts for patterns. **The universal dashboard layout.**

Default charts work. Styled charts communicate.

The built-in defaults are fine for exploration. But for a dashboard, a demo, or a presentation — spend **3 lines of code** making it look intentional. Titles, labels, colors.

✕ BEFORE · DEFAULT

3 lines of code

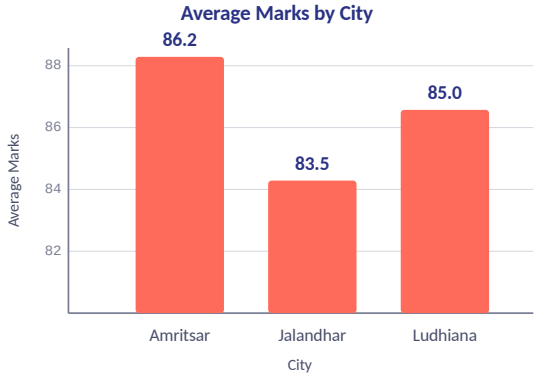


```
ax.bar(cities, averages)
```

Works. But **what is this chart about?** You can't tell without context.

✓ AFTER · STYLED

+4 lines of code




```
ax.bar(...) + ax.set_title(...) + ax.set_xlabel(...) + ax.set_ylabel(...)
+ ax.grid(...)
```

Same data. **4 extra lines.** Now it tells a story.

THE 4-LINE STYLING KIT (1) `ax.set_title()`, (2) `ax.set_xlabel()`, (3) `ax.set_ylabel()`, (4) `ax.grid(alpha=0.3)`. Add these to every Matplotlib chart. **Costs 30 seconds, doubles readability.**

CHEATSHEET · WHEN TO USE WHICH

Bookmark this. Five chart types, five use cases.

The skill is not knowing the API — it's knowing which chart fits the data. Here's the decision tree.

`st.line_chart`

USE WHEN Showing how something changes over time (test scores, sales, temperature). The x-axis is sequence or time.

DON'T You have distinct categories with no order — use a bar chart.

`st.bar_chart`

USE WHEN Comparing quantities across distinct categories (cities, branches, products). The height makes the ranking obvious.

DON'T You have more than 10 categories — the bars get cramped. Consider grouping or a horizontal bar.

`st.area_chart`

USE WHEN Showing cumulative totals or the composition of a whole over time. The filled area emphasizes volume.

DON'T You just want a line chart — area charts make exact values harder to read.

`Matplotlib (st.pyplot)`

USE WHEN You need full control — titles, labels, colors, multiple subplots, annotations, a specific style.

DON'T You just need a quick look at the data — the built-in charts are faster.

`st.metric`

USE WHEN Highlighting one important number — average, total, count, percentage. The headline of your dashboard.

DON'T You have multiple data points — use a table or chart instead.

DECISION TREE

Over time? → **`line_chart`**. Across categories? → **`bar_chart`**. One number? → **`st.metric`**.
Need styling? → **`Matplotlib`**. Volume matters? → **`area_chart`**.

03 / 03

Build a full dashboard. The Phase-2 capstone.

Today's exercise combines everything from Phase 2 — Day 5's sidebar, Day 6's pandas, Day 7's charts. The pattern every group will reuse in their project.

01 BRIEF (5 MIN)

02 BUILD (15 MIN)

03 EXTEND (5 MIN)

EXERCISE · BRIEF · 5 MIN

Build a full student dashboard. Sidebar + table + metrics + chart.

A real dashboard pattern. Controls in the sidebar (Day 5), data filtering with pandas (Day 6), KPI metrics + a bar chart (Day 7). The capstone of Phase 2.

THE BRIEF

TASK

Build a Streamlit app called `full_dashboard.py` that loads `students.csv`, displays KPI metrics + a filtered table + a bar chart of average marks by city. Sidebar filters for branch and city (from Day 6).

REQUIREMENTS

- ✓ Load the CSV with `pd.read_csv`
- ✓ Sidebar: branch selectbox + city selectbox (both with "All" option, both keyed)
- ✓ Filter the DataFrame based on selections (Day 6 pattern)
- ✓ Show 3 `st.metric` cards in columns: Average Marks, Highest Mark, Student Count
- ✓ Show the filtered DataFrame with `st.dataframe`
- ✓ Show a `st.bar_chart` of average marks by city (use `groupby`)

BY THE END

Every student has a full dashboard with sidebar + metrics + table + chart. The Phase-2 capstone pattern.

STARTING SCAFFOLD

CODE

```python
# full_dashboard.py — starting scaffold
import streamlit as st
import pandas as pd

st.title("Student Dashboard — Full")

# 1. Load data
df = pd.read_csv("students.csv")

# 2. Sidebar filters (from Day 6)
with st.sidebar:
    st.write("**Filters**")
    # TODO: branch selectbox (All + unique)
    # TODO: city selectbox (All + unique)

# 3. Filter (from Day 6)
filtered = df
# TODO: apply branch filter if not "All"
# TODO: apply city filter if not "All"

# 4. KPI metrics (Day 7)
col1, col2, col3 = st.columns(3)
# TODO: col1.metric("Avg Marks", ...)
# TODO: col2.metric("Highest", ...)
# TODO: col3.metric("Students", ...)

# 5. Filtered table
```

▶ RUN IT — `streamlit run full_dashboard.py` Pick a branch + city in the sidebar. Watch the metrics, table, and chart all update live.

EXERCISE · SOLUTION WALKTHROUGH

One way to solve it. There are others — this is just clean.

Walk through this line by line. Then go back to your own version and compare.

FULL_DASHBOARD.PY

solution · v1

```python
import streamlit as st
import pandas as pd

① st.title("Student Dashboard — Full")

② df = pd.read_csv("students.csv")

③ with st.sidebar:
    st.write("**Filters**")
    branches = ["All"] + df["branch"].unique().tolist()
    cities = ["All"] + df["city"].unique().tolist()
    branch = st.selectbox("Branch", branches, key="f_branch")
    city = st.selectbox("City", cities, key="f_city")

④ filtered = df
   if branch != "All":
       filtered = filtered[filtered["branch"] == branch]
   if city != "All":
       filtered = filtered[filtered["city"] == city]

⑤ col1, col2, col3 = st.columns(3)
   col1.metric("Avg Marks", f"{filtered['marks'].mean():.1f}")
   col2.metric("Highest", f"{filtered['marks'].max()}")
   col3.metric("Students", len(filtered))

⑥ st.write("**Filtered Students**")
   st.dataframe(filtered)

⑦ st.write("**Average Marks by City**")
```

LINE BY LINE

7 stops

① **Title**
Standard `st.title`. The page heading.

② **Load data**
`pd.read_csv` loads the CSV. One line.

③ **Sidebar filters**
Build option lists: `["All"]` + unique values. Both widgets keyed (Day 5 habit).

④ **Filter**
Start with full df, apply each filter if not "All". The universal dashboard pattern from Day 6.

⑤ **KPI metrics**
3 columns, each with `st.metric`. Format avg to 1 decimal. The dashboard headline.

⑥ **Filtered table**
`st.dataframe` renders the filtered set. Interactive — sortable, scrollable.

⑦ **Chart**
`groupby` city + mean marks, then `st.bar_chart`. Day 6 pandas + Day 7 charts = real analytics.

OTHER SOLUTIONS EXIST — You could use Matplotlib for the chart, add st.tabs for multiple views, or use st.expander for advanced filters. All valid. This version is the most readable for a beginner.

EXERCISE · EXTEND + WHAT'S NEXT

Finished early? Add a second chart. Or peek at tomorrow.

Three extension ideas for fast students. Plus a preview of Day 8 — the solo challenge that gates group formation.

EXTENSION IDEAS

FOR FAST STUDENTS

① Add a second chart — marks by branch

Below the city chart, add: `avg_by_branch = filtered.groupby("branch")["marks"].mean()` / `st.bar_chart(avg_by_branch)`. Now users see two breakdowns.

② Add a marks-range slider

In the sidebar, add `st.slider("Marks range", 0, 100, (0, 100))` and filter: `filtered = filtered[(filtered["marks"] >= low) & (filtered["marks"] <= high)]`. Combines with the existing branch/city filters.

③ Use Matplotlib for a histogram

Replace the bar chart with a Matplotlib histogram of marks distribution. `fig, ax = plt.subplots()` / `ax.hist(filtered["marks"], bins=10, color="#FF6B5B")` / `st.pyplot(fig)`. Full control over styling.

WHAT'S NEXT · DAY 8

SOLO CHALLENGE

A full app, built alone

You get a brief at the start. 90 minutes. Build it solo. No group.

The gate before groups

If you can build Day 8's app, you're ready for the group project on Day 15.

Demo to the class

At the end, you show what you built. Like a mini Demo Day.

TOMORROW — Everything you've learned in Phase 2, in one app, built by you, demoed live. Sleep well tonight.

IF YOU'RE STUCK — Pair with a neighbour. Read the error message out loud. Raise your hand. We move at the room's pace.

Q&A · CLOSING · 5 MIN

Today your data became a picture.

Tomorrow you build a full app — solo. The Day 8 challenge is the gate before group formation. Everything you've learned in Phase 2, in one app.

OPEN THE FLOOR

Ask Now

- ✓ "Which chart type felt most natural — line, bar, or Matplotlib?"
- ✓ "Did the dashboard pattern click — sidebar + metrics + table + chart?"
- ✓ "Anything from today's exercise that didn't work?"

BETWEEN SESSIONS

Get Unstuck

- ✓ **Email · `hello@zlaark.com`**
  For chart questions, paste your app.py + a screenshot of the chart.
- ✓ **Web · `www.zlaark.com`**
  Program updates and resources.
- ✓ **In-session · raise a hand**
  No question is too small. We move at the room's pace.

WHAT'S NEXT · DAY 8

Solo Challenge

- ✓ Day 8 — Build a full app alone. 90 minutes. Demo at the end.
- ✓ The gate — if you can build Day 8, you're ready for the group project.
- ✓ Phase 4 preview — Groups formed on Day 15. Project work begins.

Zlaark

See you on Day 8.